

AOS: Adaptive Domain Independent Multi AI Agent Orchestration

Hisham K H, Irfan Nazir , Harikrishnan T B , Edwin Varghese
Department of Computer Science and Engineering
Sahrdya College of Engineering and Technology
Kerala, India

I. LITERATURE SURVEY

The rapid development of large language models (LLMs) has led to the emergence of autonomous and multi-agent systems capable of reasoning, planning, task decomposition, tool utilization, and collaborative execution. Existing research has addressed these capabilities through agent orchestration frameworks, agent operating systems, model routing, failure recovery, memory systems, and interoperability protocols. However, these approaches generally address individual aspects of agent execution rather than providing a unified operating layer for capability management, scheduling, fault recovery, and workflow-level learning.

A. Agent Orchestration Frameworks

Modern multi-agent frameworks such as AutoGen, LangGraph, CrewAI, MetaGPT, and CAMEL provide abstractions for constructing and coordinating autonomous agents. These systems support role-based agents, task decomposition, message passing, and structured workflow execution. Graph-based approaches such as LangGraph enable developers to represent agent workflows using explicit states and transitions, while role-oriented systems allow multiple specialized agents to collaborate on complex tasks.

Although these frameworks provide effective orchestration primitives, resource selection and failure handling are commonly implemented through application-specific logic. The framework typically determines how agents communicate or how a workflow is structured, while the decision of which computational resource should execute a particular subtask remains outside a unified kernel-level resource abstraction. This limits the ability to perform system-wide capability-aware scheduling and dynamic resource substitution.

B. Agent Operating Systems

The concept of an Agent Operating System (Agent OS) extends traditional operating-system abstractions to intelligent agents. AIOS introduced an operating-system-inspired architecture for LLM agents, providing kernel-level services for agent scheduling, context management, tool management, and related operations.

This direction demonstrates that agent execution can benefit from centralized system-level management rather than relying entirely on application-level orchestration. However, existing Agent OS approaches remain largely LLM-centric and focus

primarily on managing language-model agents. Capability-aware scheduling across heterogeneous resources such as LLMs, vision models, speech models, tools, APIs, databases, and other computational resources remains an open problem.

Furthermore, conventional scheduling mechanisms such as FIFO or round-robin do not explicitly represent the capability requirements of individual subtasks. This creates a gap between agent-level scheduling and capability-level resource selection.

C. LLM Routing and Model Selection

A separate research direction focuses on routing requests between multiple language models. Approaches such as FrugalGPT, RouteLLM, Hybrid-LLM, and RouterBench investigate the selection of models according to factors such as quality, cost, and latency.

These approaches demonstrate that dynamically selecting a suitable model can significantly improve the efficiency of LLM-based systems. However, most routing methods operate primarily within the space of language models and commonly make routing decisions at the level of an entire user query.

An autonomous agent workflow may contain multiple subtasks with different requirements. For example, one task may require speech transcription, another may require database access, and another may require deep reasoning. Selecting a single LLM for the complete workflow is therefore insufficient for heterogeneous agent execution.

A capability-aware system must instead determine which resource can satisfy the requirements of each individual subtask while considering constraints such as cost, latency, quality, and risk.

D. Capability-Based Resource Selection

Capability-based approaches provide a mechanism for representing the skills or functions supported by computational components. Such representations can be used to determine whether a resource is technically capable of executing a particular operation.

However, existing model-routing approaches generally represent the selection problem in terms of models rather than a generalized capability space. This becomes restrictive when an agent system must select among heterogeneous resources.

AOS addresses this limitation through Capability DNA, which represents the requirements of each subtask using

discrete capability flags, ordinal complexity scores, and continuous execution constraints. The architecture therefore treats models, tools, APIs, databases, and simulated sensor or robotic interfaces as resources under a common abstraction.

E. Failure Analysis and Fault Recovery

Reliability is a significant challenge in autonomous multi-agent systems. Failures may occur during planning, reasoning, tool execution, resource access, communication, or memory retrieval. The MAST taxonomy provides a systematic classification of failures in multi-agent systems and demonstrates that agent failures can be organized into identifiable categories.

Other approaches such as Reflexion and self-refinement attempt to improve agent reliability through feedback, self-critique, and repeated execution. These techniques can improve task performance but generally treat recovery as a retry or reasoning-level process.

A limitation of existing approaches is the lack of an integrated closed-loop mechanism that detects a failure, classifies its type, and selects an appropriate recovery strategy. AOS extends this direction through a Cognitive Fault Tolerance mechanism consisting of failure detection, classification, and recovery-policy selection.

F. Agent Memory and Semantic Caching

Long-term memory has become an important component of agentic systems. Systems such as MemGPT/Letta, Mem0, and A-MEM investigate methods for retaining information across interactions and executions. Semantic caching approaches further reduce computation by reusing previous responses for semantically similar requests.

However, response-level caching does not capture the structure of an entire agent workflow. A complex workflow may contain several subtasks, resource bindings, intermediate outputs, and recovery operations that could potentially be reused.

AOS therefore introduces workflow-level experience records that store the workflow graph, Capability DNA profile, resource bindings, execution outcomes, costs, and recovery information. This provides the basis for reusing validated workflow subgraphs rather than only individual responses.

G. Experience-Driven Agent Learning

Recent research has explored methods for allowing agents to learn from previous executions. Approaches such as ExpeL and Voyager accumulate experiences or reusable skills that can improve future task execution. Prompt optimization systems such as DSPy and GEPA also demonstrate how previous execution outcomes can be used to improve agent behavior.

Although these approaches provide mechanisms for experience accumulation, they generally focus on agent-level skills, instructions, or textual knowledge. The reuse of complete, validated workflow structures together with their resource bindings remains less explored.

AOS treats workflow execution as a first-class learning artifact. The Learning Manager stores completed workflow graphs and their execution characteristics, retrieves similar

experiences, and attempts to transplant validated subgraphs into new tasks.

H. Interoperability in Agentic Systems

The increasing number of AI models and external tools has created a need for standardized interfaces. The Model Context Protocol (MCP) provides a standardized mechanism for connecting AI systems with external tools and data sources. Agent-to-agent protocols similarly provide mechanisms for communication between autonomous agents.

These protocols improve interoperability but do not themselves define how resources should be scheduled or selected. They also do not provide capability-aware feasibility analysis, admission control, or cognitive fault recovery.

AOS builds upon standardized resource interfaces while adding a kernel-level layer responsible for resource registration, capability interpretation, scheduling, monitoring, and recovery.

I. Scheduling and Resource Management

Traditional operating systems provide scheduling mechanisms that allocate computational resources according to priorities, deadlines, and resource constraints. Algorithms such as Earliest Deadline First (EDF) demonstrate how deadline-aware scheduling can be used to determine whether tasks can be executed within specified constraints.

Agentic workloads differ from conventional computational tasks because their resource requirements can include semantic capabilities, model quality, tool availability, and reasoning requirements. Consequently, conventional CPU-oriented scheduling alone cannot determine whether an agent task is executable.

AOS maps scheduling concepts into the cognitive execution domain by combining capability feasibility with cost, latency, quality, and risk constraints. This provides a foundation for capability-aware admission control and scheduling of agent workflows.

J. Research Gap

The existing literature demonstrates significant progress in agent orchestration, Agent Operating Systems, LLM routing, multi-agent failure analysis, agent memory, experience learning, and interoperability. However, these mechanisms are largely developed as separate components.

The major gaps identified from the literature are summarized below.

- 1) **Lack of a typed capability abstraction:** Existing systems do not provide a unified schedulable representation for heterogeneous resources including models, tools, APIs, databases, and simulated sensors.
- 2) **Limited modality-aware routing:** Existing routing systems primarily select between LLMs for complete queries rather than selecting heterogeneous resources for individual nodes of a task workflow.
- 3) **Open-loop failure handling:** Existing failure taxonomies describe failure types, while practical systems

commonly rely on retry or reflection. An integrated detection–classification–recovery loop is required.

- 4) **Response-level rather than workflow-level memory:** Existing caching and memory mechanisms primarily retain responses, conversations, or skills rather than complete validated workflow structures.
- 5) **Limited cognitive admission control:** Traditional scheduling concepts have not been sufficiently adapted to workloads where latency, cost, quality, and capability constraints must be satisfied simultaneously.

These gaps motivate the development of AOS as an Adaptive Cognitive AI Microkernel that integrates capability-aware resource management, scheduling, fault tolerance, and experience-driven workflow optimization within a unified orchestration layer.